

Queue: FIFO, Round Robin & BFS

A comprehensive technical reference covering Abstract Data Types, scheduling algorithms, and graph traversal — from foundational principles to production-grade implementation.

DATA STRUCTURES & ALGORITHMS

BEGINNER TO ADVANCED

Developed by **Talenciaglobal**

Topic Classification

Course Metadata

Category	Data Structures / Algorithmic Patterns
Domain	DSA / Scheduling & Graph Algorithms
Topic Type	ADT + Application Patterns
Difficulty	Beginner to Advanced

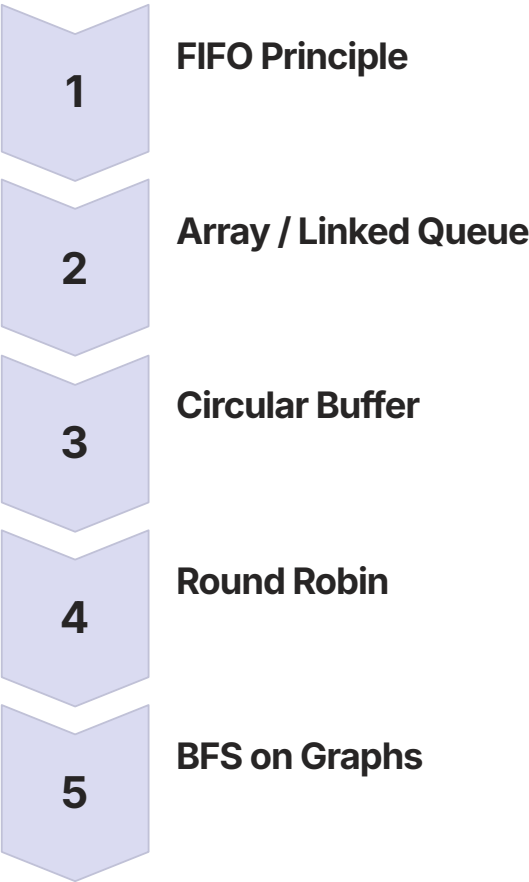
Prerequisites & Relevance

Prerequisites:

- Arrays and linked lists
- Basic recursion
- Graph representation (adjacency list)

Industry Relevance: Universal — CPU scheduling, network routers, task queues, BFS in graphs, web servers.

Recommended Learning Path



What Is a Queue?

A **queue** is an abstract data type that follows the **First In, First Out (FIFO)** discipline. Elements are inserted (*enqueued*) at the rear/tail and removed (*dequeued*) from the front/head.

FIFO

Like a supermarket line — the first person in is the first to be served. Guarantees fairness and preserves order of arrival.

Round Robin

The OS gives each program a small turn in cyclic order. When a program's turn ends, it goes to the back of the queue.

BFS

Explore all neighbours of your current location before moving further — like a wave expanding from a stone dropped in water.

Key Terminology



Enqueue

Add an element to the rear of the queue.



Dequeue

Remove and return the front element from the queue.



Front / Peek

View the front element without removing it.



Time Slice / Quantum

Maximum time a process runs before being preempted in round robin scheduling.



BFS Tree

Tree formed by parent pointers discovered during BFS traversal.



Level Order

BFS traversal order on a tree — visiting all nodes at depth k before depth $k+1$.

Why Does the Queue Exist?

Queues solve fundamental coordination problems across computing systems. The table below illustrates the contrast between unmanaged and queue-managed approaches.

Problem	Without Queue	With Queue (FIFO)
Process scheduling (OS)	Starvation, no fairness	Round robin guarantees progress
Breadth-first traversal	Recursive DFS may go deep; BFS requires level order	Queue provides natural layer-by-layer expansion
Buffer between producer and consumer	Data loss or blocking	Queue decouples rates, smooths bursts
Print spooling	Jobs printed in arbitrary order	FIFO ensures order of submission

 Industry statistic: According to Linux kernel documentation, the Completely Fair Scheduler (CFS) serves over **1 billion** Linux devices globally — all built on queue-based scheduling principles.

Industry Drivers & Real-World Motivation

Technical

Round robin is the basis of CPU schedulers, load balancers, and network routers. BFS is fundamental for shortest paths, web crawling, and social network analysis.

Business

Fair scheduling improves user experience in time-sharing systems. BFS powers "degrees of separation" on LinkedIn and Facebook, and drives recommendation algorithms.

Industry

Operating systems (Linux CFS, Windows scheduler), network switches (packet queues), and search engines (BFS for crawling) all rely on queue-based architectures.

Real-World Motivation Examples

1 CPU Scheduler

A computer runs dozens of processes; round robin gives each a 10ms slice, cycling through the ready queue continuously.

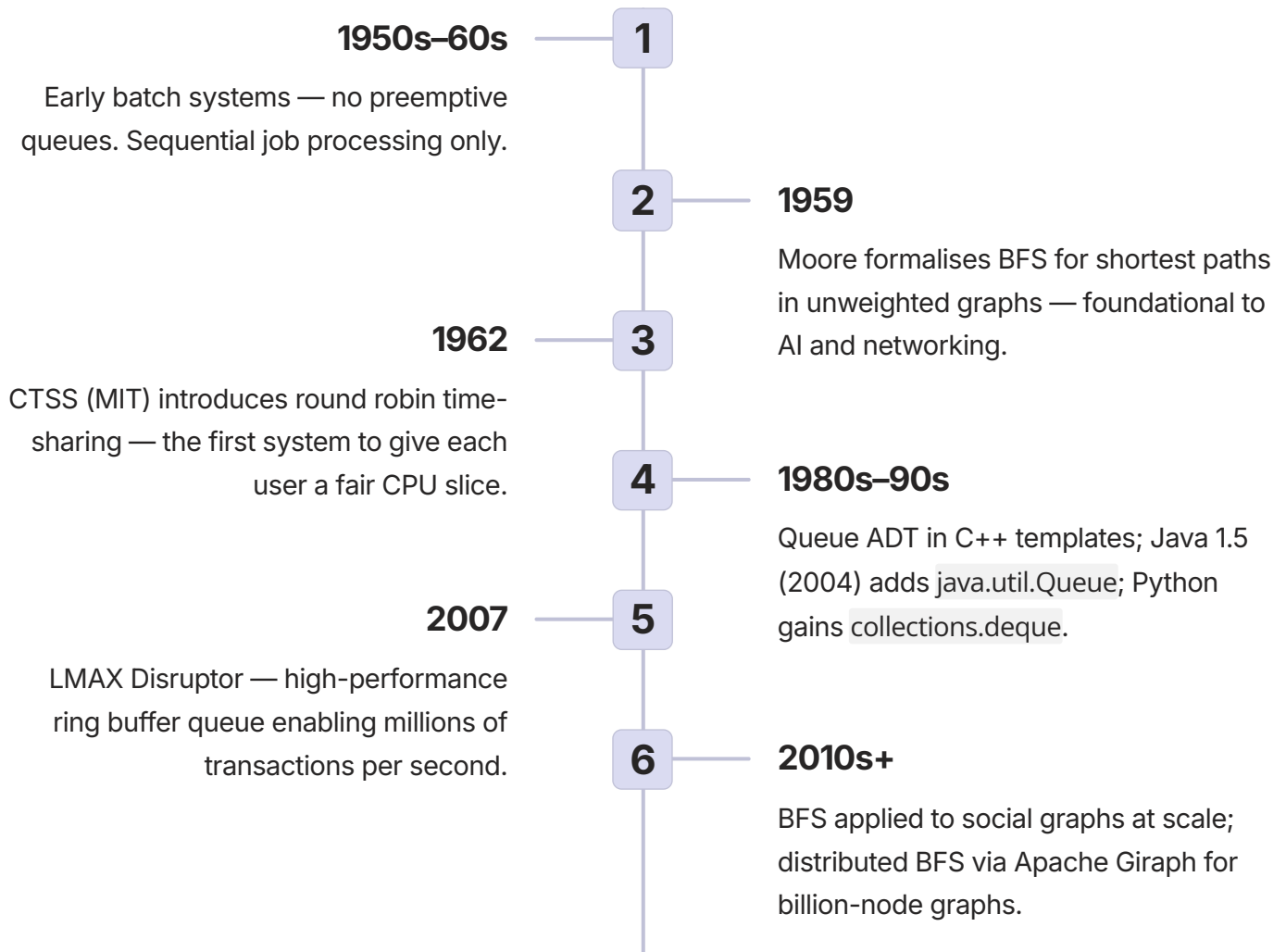
2 Web Crawler

Starting from a seed URL, links are enqueued and BFS ensures no single domain is overwhelmed — the foundation of Google's PageRank pipeline.

3 Messaging System

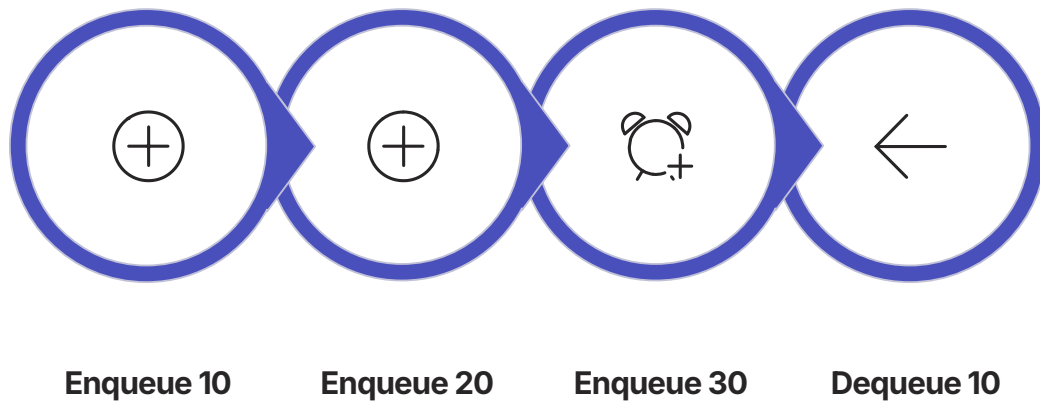
Message brokers such as RabbitMQ and Apache Kafka use queues to deliver messages in order to consumers, processing billions of messages daily.

Evolution & History



✅ Current state: Queues are part of every major standard library. Round robin remains the default scheduler for many real-time and general-purpose operating systems worldwide.

How FIFO Works: The Core Mechanism



The FIFO discipline ensures that the element inserted earliest is always the first to be removed — preserving arrival order with $O(1)$ operations at both ends.

Array-Based Circular Queue

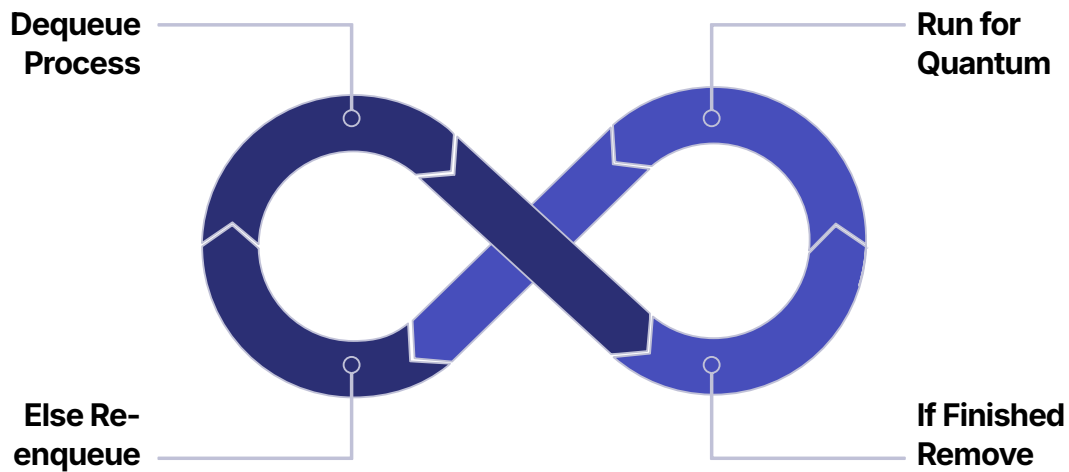
- `front` indexes the first element; `rear` points to the next free slot
- Enqueue: `arr[rear] = x; rear = (rear+1) % N`
- Dequeue: `val = arr[front]; front = (front+1) % N`
- Empty when `front == rear`
- Full when `(rear+1) % N == front`

Linked Queue (Unbounded)

- `front` pointer to first node; `rear` pointer to last node
- Enqueue: new node at `rear->next`, update `rear`
- Dequeue: move `front` forward, delete old front
- No capacity limit — grows dynamically
- Per-node allocation overhead; cache misses at scale

Round Robin Scheduling

Round robin is a preemptive scheduling algorithm that uses a queue to cycle through processes, giving each a fixed **time quantum**. It is the default scheduler in most general-purpose operating systems.



Worked Example (Quantum = 3 time units)

Queue: P1(burst=5), P2(burst=3), P3(burst=2)

Time 0: P1 runs 3 → remaining 2 → enqueue P1 → Queue: P2, P3, P1

Time 3: P2 runs 3 → finishes → Queue: P3, P1

Time 6: P3 runs 2 → finishes → Queue: P1

Time 8: P1 runs 2 → finishes → Queue: empty

Total time: 10 units

- 📌 Industry benchmark: Linux's default scheduling quantum is approximately **4–100ms** depending on system load. Studies show that a quantum of 10–20ms provides optimal throughput for interactive workloads on modern multi-core systems.

BFS: Breadth-First Search on Graphs

Algorithm (Source Vertex s)

01

Enqueue s , mark $visited[s] = true$, set $distance[s] = 0$.

02

While queue is not empty: dequeue vertex u .

03

For each unvisited neighbour v of u : mark visited, set $parent[v] = u$, $distance[v] = distance[u] + 1$, enqueue v .

04

Repeat until all reachable vertices are explored.

Level-Order Expansion

Level 0: s

Level 1: neighbours of s

Level 2: neighbours of Level 1
(not yet visited)

Level 3: neighbours of Level 2
(not yet visited)

...

Complexity

- **Time:** $O(V + E)$ with adjacency list
- **Space:** $O(V)$ for queue and visited array

Queue Variants & Types

Simple Queue (FIFO)

Basic, unbounded (linked) or bounded (array). General-purpose use across all domains.

Circular Queue

Fixed size, wraps around. Ideal for buffers and streaming data pipelines.

Deque (Double-Ended)

Insert/delete at both ends. Used for sliding window problems and palindrome checks.

Priority Queue

Elements ordered by priority (heap-backed). Powers Dijkstra's algorithm and Huffman coding.

Blocking Queue

Thread-safe; blocks when empty or full. The backbone of producer-consumer patterns (Java `BlockingQueue`).

Lock-Free Queue

Non-blocking, uses Compare-And-Swap (CAS). Designed for high-concurrency systems such as the LMAX Disruptor.

- ❏ The LMAX Disruptor ring buffer processes over **6 million transactions per second** on a single thread — demonstrating the performance ceiling of optimised lock-free queue implementations.

Real-World Use Case 1: Round Robin Load Balancer

Context

Industry	Web Services / Cloud
Problem	Distribute HTTP requests evenly across backend servers
Solution	Queue of server IPs; each request takes the front server, then server moves to back

Benefits

- Simple — no server state required
- Fair distribution across identical nodes
- $O(1)$ per request routing decision

 Works only if servers are identical. For varying capacity, use **weighted round robin**.

Python Implementation

```
from collections import deque

class RoundRobinBalancer:
    def __init__(self, servers):
        self.servers = deque(servers)

    def get_server(self):
        server = self.servers.popleft()
        self.servers.append(server)
        return server

# Usage
balancer = RoundRobinBalancer(
    ["srv1", "srv2", "srv3"]
)
for request in range(6):
    print(balancer.get_server())
```

Real-World Use Case 2: BFS for Shortest Path in Social Graphs

Java Implementation

```
public List<User> shortestPath(
    User src, User dst) {
    Queue<User> queue =
        new ArrayDeque<>();
    Map<User, User> parent =
        new HashMap<>();
    queue.add(src);
    parent.put(src, null);

    while (!queue.isEmpty()) {
        User u = queue.poll();
        if (u.equals(dst)) break;
        for (User v : u.getFriends()) {
            if (!parent.containsKey(v)) {
                parent.put(v, u);
                queue.add(v);
            }
        }
    }

    // Reconstruct path via parent map
}
```

Context & Insights

Industry	Social Media (LinkedIn, Facebook)
Problem	Find shortest connection chain between two users
Complexity	$O(V+E)$ — guarantees shortest hop count

 LinkedIn's "Degrees of Connection" feature processes BFS across a graph of over **900 million users**. At this scale, bidirectional BFS (two queues meeting in the middle) reduces exploration space by approximately **50%**.

Lesson Learned

For large graphs, use **bidirectional BFS** — two queues expanding from source and destination simultaneously, meeting in the middle.

Real-World Use Case 3: Order Processing Pipeline

Context

Industry	E-Commerce
Problem	Handle incoming order bursts asynchronously without data loss
Solution	Thread-safe blocking queue; producers add orders, consumers process them

Java Implementation

```
BlockingQueue<Order> queue =
    new LinkedBlockingQueue<>(1000);

// Producer thread
queue.put(order); // blocks if full

// Consumer thread
Order order = queue.take(); // blocks if empty
```

Benefits & Lessons

- Decoupling**
Submission and processing are fully decoupled — producers and consumers operate at independent rates.
- Load Spike Handling**
Queue absorbs burst traffic without dropping orders, smoothing demand peaks.
- Bounded Capacity**
Always use a bounded queue in production to prevent memory exhaustion under extreme load.

 Apache Kafka — a distributed queue system — processes over **1 trillion messages per day** at LinkedIn, demonstrating the industrial-scale viability of queue-based pipelines.

Implementations & Code Examples

Example 1: Generic Linked Queue (Java)

```
public class LinkedQueue<T> {
    private static class Node<T> {
        T data; Node<T> next;
        Node(T data) { this.data = data; }
    }
    private Node<T> front, rear;
    private int size = 0;

    public void enqueue(T item) {
        Node<T> newNode = new Node<>(item);
        if (rear == null) { front = rear = newNode; }
        else { rear.next = newNode; rear = newNode; }
        size++;
    }
    public T dequeue() {
        if (front == null) throw new NoSuchElementException();
        T data = front.data;
        front = front.next;
        if (front == null) rear = null;
        size--;
        return data;
    }
    public T peek() { return front == null ? null : front.data; }
    public boolean isEmpty() { return front == null; }
    public int size() { return size; }
}
```

Example 2: BFS on Grid — Shortest Path in Maze (Python)

```
from collections import deque

def bfs_maze(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    visited = [[False]*cols for _ in range(rows)]
    q = deque()
    q.append((start[0], start[1], 0)) # (r, c, distance)
    visited[start[0]][start[1]] = True
    directions = [(1,0),(-1,0),(0,1),(0,-1)]

    while q:
        r, c, dist = q.popleft()
        if (r, c) == goal:
            return dist
        for dr, dc in directions:
            nr, nc = r+dr, c+dc
            if (0 <= nr < rows and 0 <= nc < cols
                and not visited[nr][nc]
                and grid[nr][nc] != 1):
                visited[nr][nc] = True
                q.append((nr, nc, dist+1))
    return -1 # unreachable
```

Hands-On Labs



Lab 1: Round Robin Scheduler Simulator

Difficulty: Beginner

Simulate CPU scheduling with `quantum=2`. Implement the round robin loop using Python's deque. Each task is a tuple `(name, burst_time)`.

Learning Outcome: Understand preemptive scheduling and fairness guarantees.



Lab 2: BFS on Adjacency List

Difficulty: Intermediate

Implement BFS to compute distances from a source vertex across a graph represented as an adjacency list. Print each visited vertex and its distance.

Learning Outcome: Level-order traversal and distance calculation in $O(V+E)$.



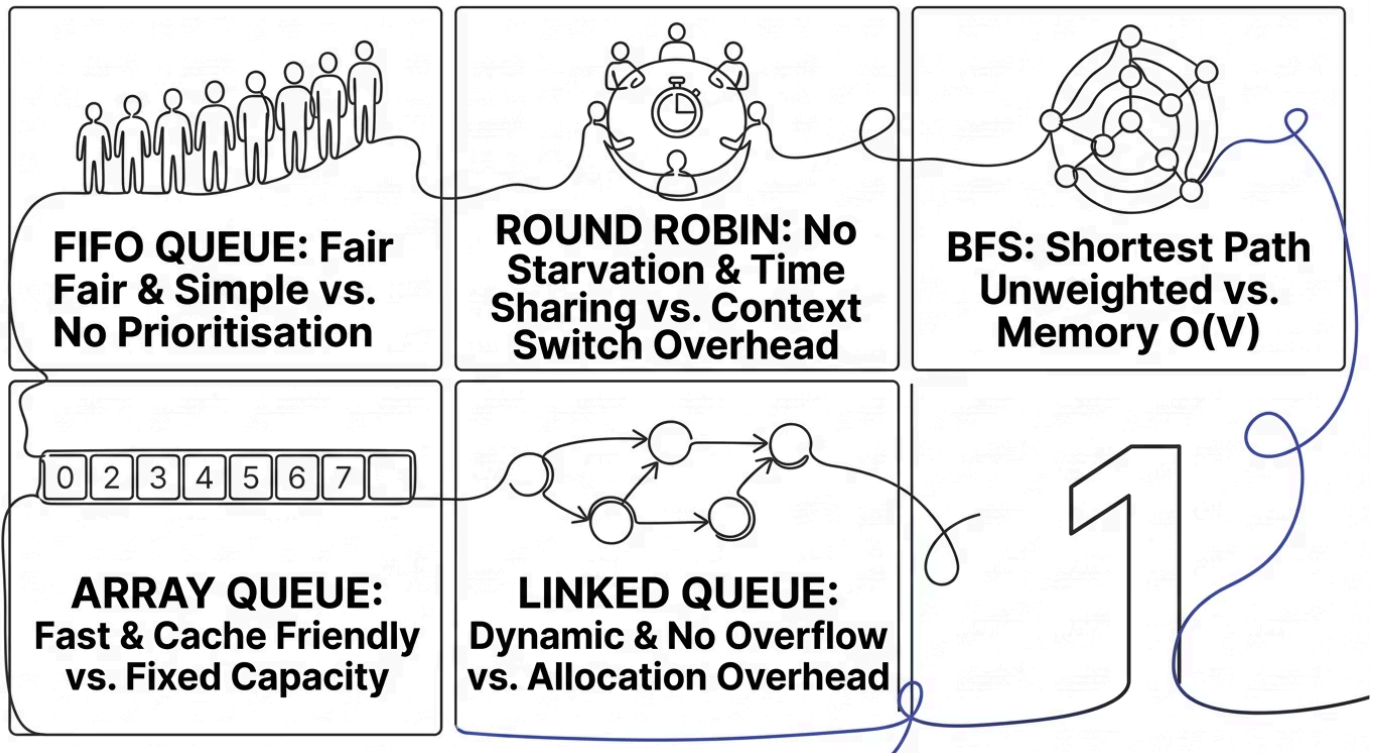
Lab 3: Sliding Window Maximum (Deque)

Difficulty: Advanced

Given array `[1,3,-1,-3,5,3,6,7]` with `k=3`, output each window's maximum using a deque of indices. Expected output: `[3,3,5,5,6,7]`.

Learning Outcome: Using deque as an efficient monotonic queue for maxima in $O(n)$.

Advantages & Disadvantages



Each queue variant presents distinct trade-offs. The optimal choice depends on workload characteristics, memory constraints, and concurrency requirements. In production systems, bounded blocking queues are the most commonly deployed variant due to their safety guarantees.

Performance Considerations

Complexity Summary

Operation	Array (Circular)	Linked
Enqueue	$O(1)$	$O(1)$
Dequeue	$O(1)$	$O(1)$
Peek	$O(1)$	$O(1)$
Space	$O(\text{capacity})$	$O(n)$ + pointer overhead

Key Benchmarks (Approximate)

- Enqueue/dequeue (circular array, C++): **5–10 ns**
- Linked queue enqueue (C++): **30–50 ns** (allocation overhead)
- BFS on 1M nodes, 2M edges (C++): **0.5–2 seconds**
- BFS on 1M nodes, 2M edges (Python): **5–10 seconds**

Optimisation Techniques



Circular Buffer

Use for fixed-size, high-throughput scenarios.



Memory Pool

Pre-allocate node pool for linked queue to eliminate allocation overhead.



Bidirectional BFS

Cut exploration space by half — critical for large social graphs.



Batch Processing

Dequeue multiple elements at once to amortise overhead.

Common Bottlenecks

Array Resizing

Non-circular growth triggers $O(n)$ copy — use circular buffer to avoid.

Cache Misses

Linked queue pointer chasing causes cache misses at large sizes.

Lock Contention

Multi-threaded queue access without lock-free design creates bottlenecks.

Memory Fragmentation

Frequent node allocation/deallocation in linked queues fragments the heap.

Scalability & Reliability

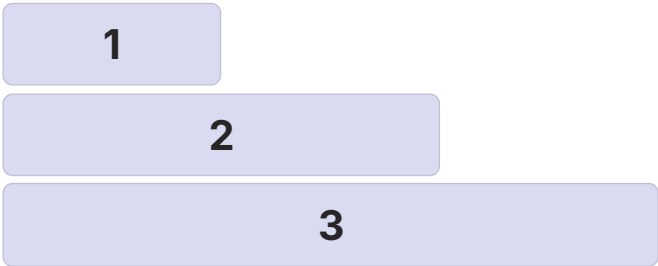
Scaling Strategies

Strategy	Queue Implementation
Vertical	Increase buffer size; use faster array-based queue
Horizontal	Partition queue (e.g., Kafka partitions); round robin across queues
Elastic	Bounded queue with backpressure; dynamic capacity (resizable ring buffer)

High Availability & Fault Tolerance

- **Persistent queue** (RabbitMQ, SQS) — messages written to disk
- **Replication** — multiple queue replicas for redundancy
- **Circuit breaker** — when queue is full, reject or fallback gracefully

Scale Progression



- 1 Small Scale**
Python `collections.deque` — simple, single-threaded, in-memory.
- 2 Medium Scale**
Java `ArrayBlockingQueue` — bounded, thread-safe, multi-producer/consumer.
- 3 Enterprise Scale**
Apache Kafka — distributed, partitioned, persistent, replicated. Handles trillions of messages per day.

Design & Architectural Considerations

Key Architectural Decisions

Decision	Options	Guidance
Bounded vs Unbounded	Fixed capacity vs unlimited	Always bounded in production to prevent memory exhaustion
Blocking vs Non-blocking	Put/take block vs poll/offer	Blocking for producer-consumer; non-blocking for low-latency systems
Single vs Multi Producer/Consumer	SPSC, MPSC, MPMC	Use concurrent queues (ConcurrentLinkedQueue, Disruptor) for MPMC
Queue Ordering	FIFO, priority, LIFO	FIFO for fairness; priority for urgency-based scheduling

BFS vs DFS: Architectural Trade-offs

BFS (Queue)	DFS (Stack)
Shortest Path (Unweighted) Yes	Shortest Path (Unweighted) No
Memory Usage $O(\text{breadth})$	Memory Usage $O(\text{depth})$
Completeness (Finite Graph) Yes	Completeness (Finite Graph) Yes
Use Cases (Examples) Web Crawling, Social Distance	Use Cases (Examples) Topological Sort, Cycle Detection
Mechanism Queue: First-In-First-Out	Mechanism Stack: Last-In-First-Out

Security Considerations

Resource Exhaustion

An unbounded queue can consume all available memory, enabling Denial-of-Service (DoS) attacks. Always enforce capacity limits on queues exposed to external input.

Race Conditions

Concurrent enqueue/dequeue without proper synchronisation leads to data corruption. Use lock-free or thread-safe implementations (`ConcurrentLinkedQueue`, `BlockingQueue`).

Data Leakage

Queue content not cleared after use (particularly in C++ arrays) may expose sensitive data. Use smart pointers and explicit clearing in unmanaged languages.

Hardening Checklist

- Always use **bounded queues** in production — set an explicit capacity limit.
- For linked queues, **validate allocation success** before proceeding.
- Clear references after dequeue in managed languages; use smart pointers in C++.
- Use **lock-free or thread-safe** implementations for all concurrent access patterns.



Best Practices

✓ DO'S

Use `collections.deque` (Python) for efficient FIFO — avoid `list.pop(0)` which is $O(n)$.

Use `ArrayDeque` (Java) instead of `LinkedList` for queue operations — significantly faster in practice.

Set a **capacity limit** for queues exposed to untrusted or external input.

In BFS, **mark visited when enqueueing** (not when dequeueing) to prevent duplicate processing.

For round robin, choose quantum carefully — too small causes overhead; too large causes unresponsiveness.

✗ DON'TS

Do **not** use `list.pop(0)` in a loop — this is $O(n^2)$ and will degrade performance catastrophically at scale.

Do **not** ignore the possibility of queue full in bounded queues — always handle rejection explicitly.

Do **not** use BFS on weighted graphs — use Dijkstra's algorithm instead for correct shortest paths.

Do **not** share a queue between threads without proper synchronisation — race conditions will corrupt state.

Common Mistakes & Pitfalls

Beginner Level

Mistake	Fix
Using <code>list.pop(0)</code> as queue ($O(n)$ per pop)	Use <code>deque</code> or implement circular buffer
Forgetting to update <code>rear</code> in linked enqueue	Set <code>rear.next</code> and move <code>rear</code> pointer
Off-by-one in circular queue full condition	Use <code>(rear+1)%capacity == front</code> or maintain a count variable

Intermediate Level

Mistake	Fix
BFS not marking visited before enqueue	Mark when adding to queue — not when processing — to avoid $O(V^2)$ worst case
Round robin quantum too small	Profile and adjust; typical range is 10–100ms for interactive systems
Not handling <code>null</code> returns from <code>poll()</code>	<code>poll()</code> returns <code>null</code> on empty queue — always handle accordingly

Advanced Level

Mistake	Fix
ABA problem in lock-free queue memory reclamation	Use hazard pointers or epoch-based reclamation
False sharing in concurrent ring buffer	Pad elements to cache line size (64 bytes)
BFS on extremely deep graph with large branching factor	Use bidirectional BFS or iterative deepening DFS to reduce memory

Comparison with Alternatives

Queue Variants vs Other Data Structures

Feature	FIFO Queue	Priority Queue	Stack (LIFO)	Deque
Ordering	Arrival time	Priority	Reverse arrival	Both ends
Scheduling use	Round robin	SJF, deadline	Not typical	Not typical
Graph traversal	BFS	Dijkstra	DFS	BFS/DFS both
Insert complexity	$O(1)$	$O(\log n)$	$O(1)$	$O(1)$
Delete complexity	$O(1)$	$O(\log n)$	$O(1)$	$O(1)$

When to Use BFS vs Other Shortest Path Algorithms

Algorithm	Graph Type	Data Structure
BFS	Unweighted	Queue
Dijkstra	Non-negative weights	Priority queue (min-heap)
Bellman-Ford	Negative weights (no negative cycles)	Relaxation (no queue needed)
A*	Heuristic + weights	Priority queue

Learning Summary & Key Takeaways

$O(1)$

All Queue Ops

Enqueue, dequeue, and peek are all constant time for both array and linked implementations.

$O(V+E)$

BFS Time

BFS on a graph with V vertices and E edges using an adjacency list representation.

$O(V)$

BFS Space

Space required for the queue and visited array during BFS traversal.

7

Core Takeaways

Fundamental principles every practitioner must internalise before advancing to concurrent and distributed queues.

1 Queue = FIFO

Fair, preserves order of arrival — the foundational guarantee of all queue-based systems.

2 Round Robin

Uses a queue to cycle tasks; ensures no process starves — the basis of most OS schedulers.

3 BFS

Uses a queue to explore nodes level by level; finds shortest path in unweighted graphs in $O(V+E)$.

4 Circular Buffer

The most efficient fixed-size queue — cache-friendly, $O(1)$ operations, no allocation overhead.

5 Concurrent Queues

Essential for multi-threaded systems — choose blocking, lock-free, or partitioned based on throughput requirements.

Common Interview Questions

1

Queue Using Two Stacks

Enqueue $O(1)$, dequeue amortised $O(1)$. Use one stack for input, one for output — transfer only when output stack is empty.

2

Round Robin & Starvation

Every process eventually gets the CPU because it cycles through the queue — no process is permanently left behind.

3

BFS Shortest Path Proof

BFS explores all nodes at distance d before any node at distance $d+1$ — guaranteeing the first path found is shortest.

4

Circular Queue Full Condition

One slot is left empty to distinguish full from empty. Full: $(\text{rear}+1)\%N == \text{front}$. Alternatively, maintain a count variable.

5

BFS vs DFS

BFS uses a queue, explores neighbours first, finds shortest path. DFS uses a stack/recursion, goes deep first, uses less memory on deep graphs.

6

Bounded Blocking Queue

Use a mutex and condition variables: wait when empty for dequeue, wait when full for enqueue. Java's `LinkedBlockingQueue` implements this pattern.

❓ Q7: Can BFS detect cycles in an undirected graph? **Yes.** If you encounter an already-visited node that is not the parent of the current node, a cycle exists.

Revision Cheat Sheet

FIFO Queue Operations

enqueue(x) → insert at rear
dequeue() → remove from front
peek() → view front (no remove)

Circular Array:

front = (front+1) % N

rear = (rear+1) % N

Empty: front == rear

Full: (rear+1)%N == front

Linked List:

front & rear pointers

Enqueue: rear.next = new; rear = new

Dequeue: front = front.next

Round Robin Pseudocode

```
while queue not empty:
    process = queue.dequeue()
    run process for quantum
    if not finished:
        queue.enqueue(process)
```

BFS Pseudocode (Unweighted Graph)

```
queue.enqueue(start)
visited[start] = true
distance[start] = 0

while queue not empty:
    u = queue.dequeue()
    for v in neighbours[u]:
        if not visited[v]:
            visited[v] = true
            distance[v] = distance[u] + 1
            parent[v] = u
            queue.enqueue(v)
```

Complexity Summary

All queue ops: $O(1)$
BFS time: $O(V + E)$
BFS space: $O(V)$
Round robin: $O(n * \text{burst/quantum})$

Memory Aids

- **FIFO** — "Queue = line at a counter"
- **Round Robin** — "Each gets a turn, then back in line"
- **BFS** — "Expand the frontier wave by wave"

Recommended Next Topics



Priority Queues (Heaps)

The natural extension of FIFO queues — elements are ordered by priority rather than arrival time. Essential for Dijkstra's algorithm, Huffman coding, and event-driven simulation.



Deque & Sliding Window

Advanced queue applications including the sliding window maximum problem, monotonic deque patterns, and palindrome detection — all $O(n)$ solutions to problems that naively require $O(n^2)$.



Concurrent Data Structures

Producer-consumer patterns, lock-free queues (CAS-based), the LMAX Disruptor ring buffer, and NUMA-aware queue design for multi-socket systems.



Graph Algorithms

Topological sort (Kahn's algorithm uses a queue), Strongly Connected Components, and distributed BFS on billion-node graphs using Apache Giraph or Spark GraphX.

"The queue is one of the most fundamental abstractions in computer science — mastering it unlocks understanding of operating systems, networking, graph theory, and distributed systems simultaneously."

— **Talenciaglobal** DSA Curriculum Team